

Scanning using Nmap and Wireshark

By: Oluwatoni Jason Oderinlo

LinkedIn: [linkedin.com/in/toni-oderinlo](https://www.linkedin.com/in/toni-oderinlo)

Agenda

Introduction about project

Introduction about tools:

- Introduction about Wireshark
- Introduction about Nmap and NSE

Project requirement and testing website/network

Perform Wireshark IDS/Firewall evade techniques (3 techniques)

Perform NSE scanning for testing network/website (3 scripts)

Conclusion

Reference

Thank You



WARNING

The information provided in this presentation is only intended for educational purpose only.

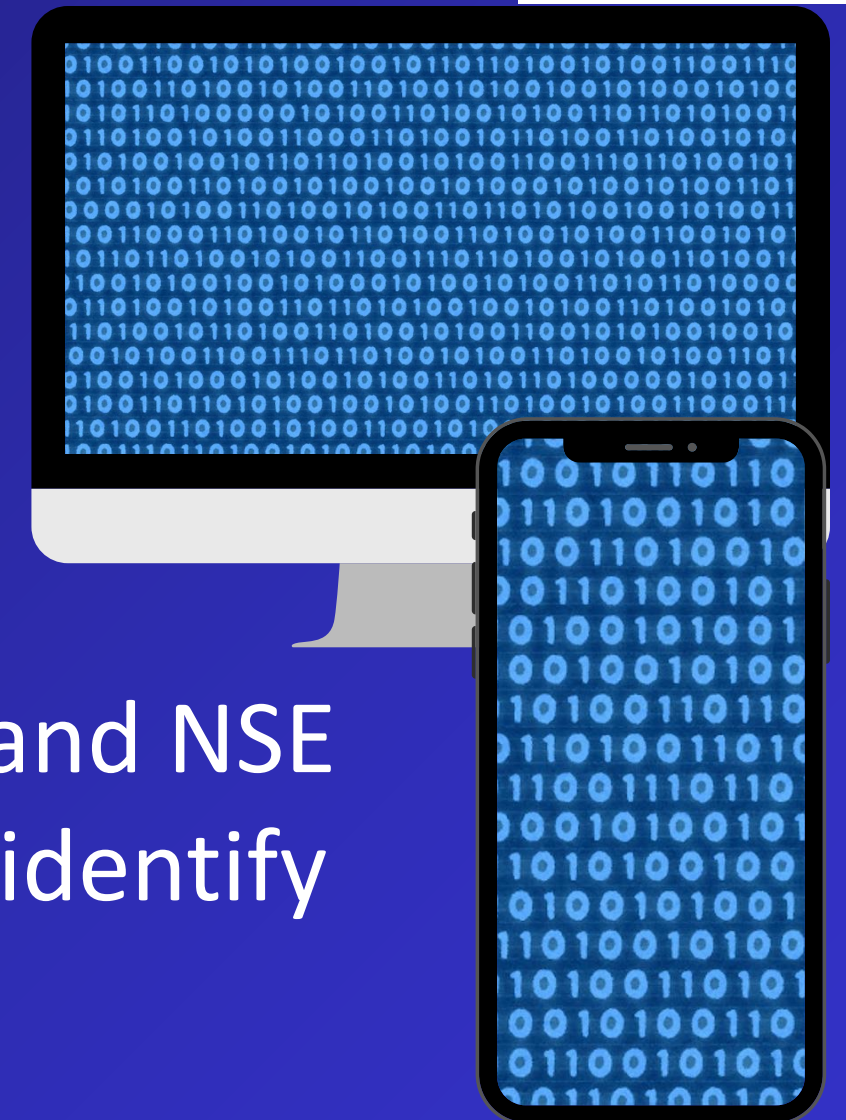
"Running scans or exploits on networks may cause unintended disruptions or service outages. Ensure you have proper authorization and proceed with caution to avoid impacting critical systems.

Any collected data during testing must be handled with strict confidentiality and used only for the purposes outlined in this project.



Introduction about the project

- The goal of this project is to investigate the methods of network and website scanning using Nmap, Wireshark and NSE Scripts. Discovering vulnerabilities and learning how to identify the IDS and firewalls are the project's main goals.



Introduction about Tools used

- Wireshark: This an open-source network protocol analyzer used to capture, record and inspect data packets that travel across a network. It will analyze network traffic and demonstrate how particular Nmap evasion techniques might evade the firewall or Intrusion detection systems.
- Nmap and NSE: Network Mapper(Nmap) is an open-source network scanning tool that is used to discover hosts, services and vulnerabilities within a network. Nmap Scripting engine(NSE) enables users to run custom scripts for tasks like vulnerability identification, service enumeration and more.



Project Requirement's and testing website/network

Tools: Nmap, Wireshark, testing websites, window system and kali
Virtual Machine – Setup Kali Linux and Windows machine using a VM
Virtual Box

Windows Server IP: 10.0.0.102

Kali Linux: 10.0.0.222 (Test Machine)

Test Website: test.vulnweb.com, hackthissite.com



Perform Wireshark IDS/Firewall evade techniques.

I will perform 3 Wireshark IDS/Firewall evasion techniques such as Packet Fragmentation, IP Address Spoofing, Source Port Manipulation

- Packet Fragmentation - The IP packets are broken into smaller pieces to ensure that it can be transmitted across a network. Fragmentation can also be used as an evasion technique to bypass intrusion detection systems (IDS) or firewalls, as fragmented packets may not be analyzed effectively if the system fails to properly reassemble them for inspection
- IP Address Spoofing - It falsifies the source IP address of the packets to appear to originate from a different system. It's mainly used to bypass security mechanisms like firewall, intrusion detection system (IDS), or logging tools that rely on the source IP address for detecting and blocking.
- Source Port Manipulation – It modifies where the source port are from well-know ports such as HTTP, DNS, FTP and others of its packet to bypass firewalls or other security mechanisms. By making the packets look legitimate it's able to obscure the scan's intent of looking for threats.



Packet Fragmentation

-f flag is used to create packet fragments.
As a result, the packet is divided into pieces no larger than 8 bytes after the IP Header.

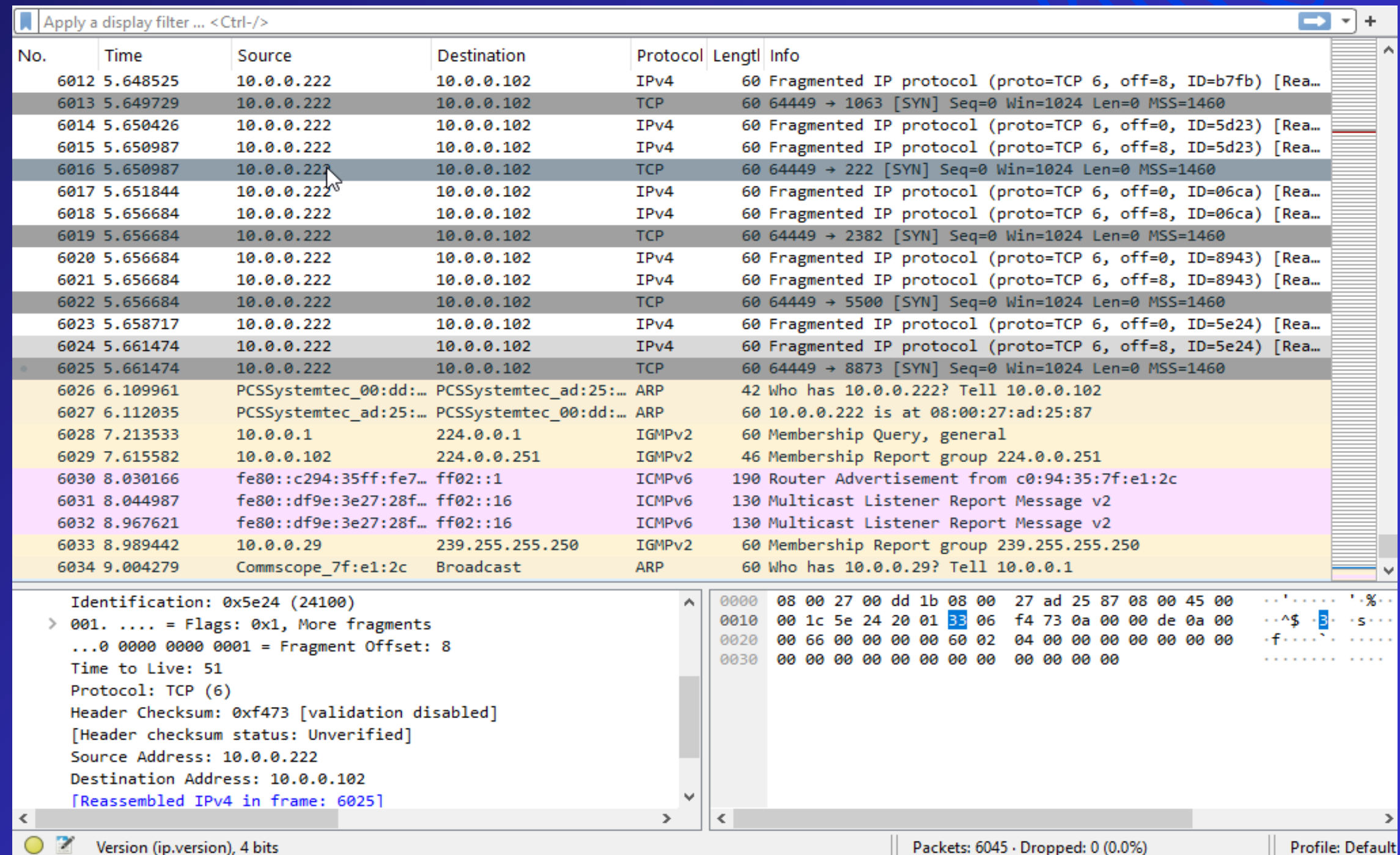
```
(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
└─(root㉿kali)-[/home/kali]
└─# nmap -f 10.0.0.102
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-19 09:59 EST
Nmap scan report for WIN-SFA18PMNQFM.phub.net.cable.rogers.com (10.0.0.102)
Host is up (0.0068s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsdapi
MAC Address: 08:00:27:00:DD:1B (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 29.60 seconds

└─(root㉿kali)-[/home/kali]
└─#
```

Fig.1 displays the output of the command for Packet Fragmentation to the target network.

In Fig.2 shows the fragmented packets gotten from the network when using the command in fig.1 on Wireshark.



No.	Time	Source	Destination	Protocol	Length	Info
6012	5.648525	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=8, ID=b7fb) [Rea...
6013	5.649729	10.0.0.222	10.0.0.102	TCP	60	64449 → 1063 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6014	5.650426	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=0, ID=5d23) [Rea...
6015	5.650987	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=8, ID=5d23) [Rea...
6016	5.650987	10.0.0.222	10.0.0.102	TCP	60	64449 → 222 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6017	5.651844	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=0, ID=06ca) [Rea...
6018	5.656684	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=8, ID=06ca) [Rea...
6019	5.656684	10.0.0.222	10.0.0.102	TCP	60	64449 → 2382 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6020	5.656684	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=0, ID=8943) [Rea...
6021	5.656684	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=8, ID=8943) [Rea...
6022	5.656684	10.0.0.222	10.0.0.102	TCP	60	64449 → 5500 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6023	5.658717	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=0, ID=5e24) [Rea...
6024	5.661474	10.0.0.222	10.0.0.102	IPv4	60	Fragmented IP protocol (proto=TCP 6, off=8, ID=5e24) [Rea...
6025	5.661474	10.0.0.222	10.0.0.102	TCP	60	64449 → 8873 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
6026	6.109961	PCSSystemtec_00:dd:...	PCSSystemtec_ad:25:...	ARP	42	Who has 10.0.0.222? Tell 10.0.0.102
6027	6.112035	PCSSystemtec_ad:25:...	PCSSystemtec_00:dd:...	ARP	60	10.0.0.222 is at 08:00:27:ad:25:87
6028	7.213533	10.0.0.1	224.0.0.1	IGMPv2	60	Membership Query, general
6029	7.615582	10.0.0.102	224.0.0.251	IGMPv2	46	Membership Report group 224.0.0.251
6030	8.030166	fe80::c294:35ff:fe7...	ff02::1	ICMPv6	190	Router Advertisement from c0:94:35:7f:e1:2c
6031	8.044987	fe80::df9e:3e27:28f...	ff02::16	ICMPv6	130	Multicast Listener Report Message v2
6032	8.967621	fe80::df9e:3e27:28f...	ff02::16	ICMPv6	130	Multicast Listener Report Message v2
6033	8.989442	10.0.0.29	239.255.255.250	IGMPv2	60	Membership Report group 239.255.255.250
6034	9.004279	Commscope_7f:e1:2c	Broadcast	ARP	60	Who has 10.0.0.29? Tell 10.0.0.1

Identification: 0x5e24 (24100)
 > 001. = Flags: 0x1, More fragments
 ...0 0000 0000 0001 = Fragment Offset: 8
 Time to Live: 51
 Protocol: TCP (6)
 Header Checksum: 0xf473 [validation disabled]
 [Header checksum status: Unverified]
 Source Address: 10.0.0.222
 Destination Address: 10.0.0.102
 [Reassembled IPv4 in frame: 6025]

Version (ip.version), 4 bits

Packets: 6045 · Dropped: 0 (0.0%)

Profile: Default

Fig.2

IP Address Spoofing

–sT –Pn –spoof-mac 0 represents randomizing the MAC address, -sT: performs the TCP connect/full open scan, -Pn is used to skip the host discovery

Fig.3 displays the nmap output of the IP address spoofing to the target machine.

Fig.4.1 and 4.2 displays the output of the command on Wireshark on the target machine. It performs a full TCP connect scan, how it skips the host discover and the randomization of MAC address

```
(root@kali)-[/home/kali]
# nmap -sT -Pn -spoof-mac 0 10.0.0.102

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-19 17:13 EST
Spoofing MAC address BA:E0:C6:F7:CA:B4 (No registered vendor)
You have specified some options that require raw socket access.
These options will not be honored for TCP Connect scan.
Stats: 0:00:05 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 2.00% done; ETC: 17:17 (0:04:05 remaining)
Stats: 0:00:07 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 3.00% done; ETC: 17:17 (0:03:46 remaining)
Stats: 0:00:33 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 16.00% done; ETC: 17:16 (0:02:53 remaining)
Stats: 0:00:42 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 20.50% done; ETC: 17:16 (0:02:43 remaining)
Stats: 0:00:57 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 28.00% done; ETC: 17:16 (0:02:27 remaining)
Stats: 0:01:13 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 36.00% done; ETC: 17:16 (0:02:10 remaining)
Nmap scan report for WIN-SFA18PMNQFM.phub.net.cable.rogers.com (10.0.0.102)
Host is up (0.010s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsddapi

Nmap done: 1 IP address (1 host up) scanned in 113.00 seconds

(root@kali)-[/home/kali]
```

Fig.3

No.	Time	Source	Destination	Protocol	Lengt	Info
2403	110.223275	52.167.164.252	10.0.0.102	TCP	66	443 → 49708 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 M...
2404	110.223275	10.0.0.222	10.0.0.102	TCP	74	57128 → 8652 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2405	110.223425	10.0.0.102	52.167.164.252	TCP	54	49708 → 443 [ACK] Seq=1 Ack=1 Win=263424 Len=0
2406	110.228640	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 58958 → 3031 [SYN] Seq=0 Win=32120 L...
2407	110.230260	10.0.0.102	52.167.164.252	TLSv1.2	269	Client Hello (SNI=geo.prod.do.dsp.mp.microsoft.com)
2408	110.234776	10.0.0.222	10.0.0.102	TCP	74	54432 → 4004 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2409	110.240242	10.0.0.222	10.0.0.102	TCP	74	59172 → 1641 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2410	110.240242	10.0.0.222	10.0.0.102	TCP	74	53278 → 2998 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2411	110.240242	10.0.0.222	10.0.0.102	TCP	74	40258 → 6100 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2412	110.240242	10.0.0.222	10.0.0.102	TCP	74	32786 → 1061 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2413	110.242385	10.0.0.222	10.0.0.102	TCP	74	58962 → 3031 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2414	110.246477	10.0.0.222	10.0.0.102	TCP	74	41818 → 6 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM ...
2415	110.246477	10.0.0.222	10.0.0.102	TCP	74	54602 → 5810 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2416	110.248734	10.0.0.222	10.0.0.102	TCP	74	56660 → 5718 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
2417	110.269610	52.167.164.252	10.0.0.102	TCP	1514	443 → 49708 [ACK] Seq=1 Ack=216 Win=525312 Len=1460 [TCP ...
2418	110.271458	52.167.164.252	10.0.0.102	TLSv1.2	1025	Server Hello, Certificate, Server Key Exchange, Server He...
2419	110.271651	10.0.0.102	52.167.164.252	TCP	54	49708 → 443 [ACK] Seq=216 Ack=2432 Win=263424 Len=0
2420	110.293469	10.0.0.102	52.167.164.252	TLSv1.2	212	Client Key Exchange, Change Cipher Spec, Encrypted Handsh...
2421	110.304085	10.0.0.102	64.71.255.204	DNS	89	Standard query 0xc3b3 AAAA v10.events.data.microsoft.com
2422	110.304213	10.0.0.102	64.71.255.204	DNS	89	Standard query 0xa106 A v10.events.data.microsoft.com
2423	110.326350	64.71.255.204	10.0.0.102	DNS	271	Standard query response 0xc3b3 AAAA v10.events.data.micro...
2424	110.326350	64.71.255.204	10.0.0.102	DNS	233	Standard query response 0xa106 A v10.events.data.microsof...
2425	110.333689	52.167.164.252	10.0.0.102	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message

> Frame 2406: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
 > Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: PCS...
 > Internet Protocol Version 4, Src: 10.0.0.222, Dst: 10.0.0.102
 > Transmission Control Protocol, Src Port: 58958, Dst Port: 3031, Seq: 58958, Win: 32120, Len: 0

Version (ip.version), 4 bits Packets: 3794 · Dropped: 0 (0.0%) Profile: Default

No.	Time	Source	Destination	Protocol	Lengt	Info
1016	86.173868	10.0.0.222	10.0.0.102	TCP	74	36932 → 5877 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1017	86.173868	10.0.0.222	10.0.0.102	TCP	74	36908 → 9071 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1018	86.173868	10.0.0.222	10.0.0.102	TCP	74	40836 → 1521 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1019	86.173868	10.0.0.222	10.0.0.102	TCP	74	40194 → 45100 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_P...
1020	86.173868	10.0.0.222	10.0.0.102	TCP	74	50652 → 32773 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_P...
1021	86.173868	10.0.0.222	10.0.0.102	TCP	74	46782 → 20 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM...
1022	86.173868	10.0.0.222	10.0.0.102	TCP	74	35290 → 1027 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1023	86.173868	10.0.0.222	10.0.0.102	TCP	74	53670 → 6004 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1024	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 53670 → 6004 [SYN] Seq=0 Win=32120 L...
1025	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 35290 → 1027 [SYN] Seq=0 Win=32120 L...
1026	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 46782 → 20 [SYN] Seq=0 Win=32120 Len...
1027	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 50652 → 32773 [SYN] Seq=0 Win=32120 ...
1028	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 40194 → 45100 [SYN] Seq=0 Win=32120 ...
1029	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 40836 → 1521 [SYN] Seq=0 Win=32120 L...
1030	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 36908 → 9071 [SYN] Seq=0 Win=32120 L...
1031	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 36932 → 5877 [SYN] Seq=0 Win=32120 L...
1032	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 33204 → 5907 [SYN] Seq=0 Win=32120 L...
1033	87.170957	10.0.0.222	10.0.0.102	TCP	74	[TCP Retransmission] 50394 → 5101 [SYN] Seq=0 Win=32120 L...
1034	87.172248	10.0.0.222	10.0.0.102	TCP	74	34700 → 2382 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1035	87.172248	10.0.0.222	10.0.0.102	TCP	74	49608 → 89 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM...
1036	87.172248	10.0.0.222	10.0.0.102	TCP	74	56974 → 1840 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1037	87.173697	10.0.0.222	10.0.0.102	TCP	74	40278 → 1000 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...
1038	87.173697	10.0.0.222	10.0.0.102	TCP	74	38222 → 1058 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PE...

> Frame 1024: 74 bytes on wire (592 bits), 74 bytes captured (592 bit...
 > Ethernet II, Src: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87), Dst: F...
 > Destination: PCSSystemtec_00:dd:1b (08:00:27:00:dd:1b)
 > Source: PCSSystemtec_ad:25:87 (08:00:27:ad:25:87)
 Type: IPv4 (0x0800)
 [Stream index: 0]
 > Internet Protocol Version 4, Src: 10.0.0.222, Dst: 10.0.0.102
 0100 = Version: 4
 0101 = Header Length: 20 bytes (5)
 > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Version (ip.version), 4 bits Packets: 3794 · Dropped: 0 (0.0%) Profile: Default

Fig.4.1

Fig.4.2

Source Port Manipulation

–g or --source-port flag options perform source port manipulation

```
(root@kali)-[/home/kali]
# nmap -g 80 10.0.0.102

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-19 17:22 EST
Stats: 0:00:02 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 5.00% done; ETC: 17:23 (0:00:38 remaining)
Nmap scan report for WIN-SFA18PMNQFM.phub.net.cable.rogers.com (10.0.0.102)
Host is up (0.0011s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsdapi
MAC Address: 08:00:27:00:DD:1B (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 18.18 seconds

(root@kali)-[/home/kali]
```

Fig.5

Fig.6 demonstrates how the network perceives the source ports as legitimate when targeting the machine, as it shows the source port being set to 80, a well-known port, leading to it being mistaken for legitimate traffic.

No.	Time	Source	Destination	Protocol	Length	Info
19	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 80 [SYN] Seq=0 Win=1024 Len=0
20	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 111 [SYN] Seq=0 Win=1024 Len=0
21	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 1720 [SYN] Seq=0 Win=1024 Len=0
22	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 445 [SYN] Seq=0 Win=1024 Len=0
23	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 135 [SYN] Seq=0 Win=1024 Len=0
24	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 110 [SYN] Seq=0 Win=1024 Len=0
25	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 993 [SYN] Seq=0 Win=1024 Len=0
26	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 8888 [SYN] Seq=0 Win=1024 Len=0
27	3.179385	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 554 [SYN] Seq=0 Win=1024 Len=0
28	3.277406	10.0.0.222	10.0.0.102	TCP	60	80 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
29	3.277406	10.0.0.222	10.0.0.102	TCP	60	80 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	3.277406	10.0.0.222	10.0.0.102	TCP	60	80 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
31	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
33	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
34	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
36	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
37	3.281076	10.0.0.222	10.0.0.102	TCP	60	80 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	3.383695	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 3306 [SYN] Seq=0 Win=1024 Len=0
39	3.383695	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 25 [SYN] Seq=0 Win=1024 Len=0
40	3.383695	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 256 [SYN] Seq=0 Win=1024 Len=0
41	3.383695	10.0.0.222	10.0.0.102	TCP	60	[TCP Port numbers reused] 80 → 143 [SYN] Seq=0 Win=1024 Len=0

Transmission Control Protocol, Src Port: 80, Dst Port: 135, Seq: 0, Len: 0	
Source Port: 80	
Destination Port: 135	
[Stream index: 15]	
> [Conversation completeness: Incomplete, SYN_SENT (1)]	
[TCP Segment Len: 0]	
Sequence Number: 0 (relative sequence number)	
Sequence Number (raw): 625295672	
[Next Sequence Number: 1 (relative sequence number)]	
Acknowledgment Number: 0	

0000	08 00 27 00 dd 1b 08 00 27 ad 25 87 08 00 45
0010	00 2c 6a 14 00 00 30 06 0b 75 0a 00 00 de 0a
0020	00 66 00 50 00 87 25 45 41 38 00 00 00 00 60
0030	04 00 17 8f 00 00 02 04 05 b4 00 00

wireshark_EthernetKAAWY2.pcapng Packets: 2118 · Dropped: 0 (0.0%) Profile: Default

Fig.6

Perform NSE scanning for testing network/website (3 scripts)

Vulnerability Scanning using common vulnerability scanning scripts

The NSE scripts used for this project were http-headers.NSE, dns-brute.NSE and http-vuln-cve2017-5638.nse

Fig.7 shows the output of http-header.nse script running on the test website

```
(root@kali)-[/home/kali]
# nmap -script http-headers test.vulnweb.com

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-19 17:43 EST
Stats: 0:00:08 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 99.99% done; ETC: 17:43 (0:00:00 remaining)
Nmap scan report for test.vulnweb.com (44.228.249.3)
Host is up (0.092s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 9.37 seconds
```

Fig.7

Fig.8 illustrates the content of the http-headers.nse script and provides an example of the expected output or a similar result when executing the script with Nmap against the target test.vulnweb.com

```
1|local http = require "http"
2|local shortport = require "shortport"
3|local stdnse = require "stdnse"
4|local table = require "table"
5
6|description = [[
7|Performs a HEAD request for the root folder ("/") of a web server and displays the HTTP headers returned.
8|]]
9
10|—
11|-- @output
12|-- PORT  STATE SERVICE
13|-- 80/tcp open  http
14|-- | http-headers:
15|-- |   Date: Fri, 25 Jan 2013 17:39:08 GMT
16|-- |   Server: Apache/2.2.14 (Ubuntu)
17|-- |   Accept-Ranges: bytes
18|-- |   Vary: Accept-Encoding
19|-- |   Connection: close
20|-- |   Content-Type: text/html
21|-- |
22|-- |_ (Request type: HEAD)
23|--
```

Fig.8

Perform NSE scanning for testing network/website

DNS Scanning

In Fig.9 shows to output when using the dns-brute.NSE script to brute force attack the target system.,

```
(root@kali)-[/home/kali] $  
# nmap -script dns-brute 10.0.0.102  
42 -- | DNS Brute-force hostnames  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-19 17:37 EST  
Stats: 0:00:05 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan  
SYN Stealth Scan Timing: About 19.50% done; ETC: 17:38 (0:00:21 remaining)  
Nmap scan report for WIN-SFA18PMNQFM.phub.net.cable.rogers.com (10.0.0.102)  
Host is up (0.0014s latency).  
Not shown: 999 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
5357/tcp  open  wsapi  
MAC Address: 08:00:27:00:DD:1B (Oracle VirtualBox virtual NIC)  
51 -- <table key="DNS Brute-force hostnames">  
Host script results:  
| dns-brute:  
|   DNS Brute-force hostnames:  
|54 --   www.phub.net.cable.rogers.com - 66.46.148.83  
|55 --   smtp.phub.net.cable.rogers.com - 66.218.88.162  
56 -- </table>  
Nmap done: 1 IP address (1 host up) scanned in 29.76 seconds
```

Fig.9

In Fig.10 the dns-brute.nse script performs a dns-brute force attack into the target system to discover the subdomains associated with it. It displays valid subdomains that responded to the DNS query.

```
13 description = [[
14 Attempts to enumerate DNS hostnames by brute force guessing of common
15 subdomains. With the <code>dns-brute.srv</code> argument, dns-brute will also
16 try to enumerate common DNS SRV records.
17
18 Wildcard records are listed as "*A" and "*AAAA" for IPv4 and IPv6 respectively.
19 ]]
20 -- 2011-01-26
21
22 --
23 -- @usage
24 -- nmap --script dns-brute --script-args dns-brute.domain=foo.com,dns-brute.threads=6,dns-brute.hostlist=./hostfile.txt,newtargets -sS -p 80
25 -- nmap --script dns-brute www.foo.com
26 -- @args dns-brute.hostlist The filename of a list of host strings to try.
27 -- Defaults to "nselib/data/vhosts-default.lst"
28 -- @args dns-brute.threads Thread to use (default 5).
29 -- @args dns-brute.srv Perform lookup for SRV records
30 -- @args dns-brute.srvlist The filename of a list of SRV records to try.
31 -- Defaults to "nselib/data/dns-srv-names"
32 -- @args dns-brute.domain Domain name to brute force if no host is specified
33 --
34 -- @see dns-nsec3-enum.nse
35 -- @see dns-ip6-arpa-scan.nse
36 -- @see dns-nsec-enum.nse
37 -- @see dns-zone-transfer.nse
38 --
39 -- @output
40 -- Pre-scan script results:
41 -- | dns-brute:
42 -- |   DNS Brute-force hostnames
43 -- |   www.foo.com - 127.0.0.1
44 -- |   mail.foo.com - 127.0.0.2
45 -- |   blog.foo.com - 127.0.1.3
46 -- |   ns1.foo.com - 127.0.0.4
47 -- |   admin.foo.com - 127.0.0.5
48 -- |_   *A: 127.0.0.123
49 --
```

Fig.10

Perform NSE scanning for testing network/website

Example of target machine with http-header

Fig.11 displays the output of running the http-vuln-cve2017-5638.nse running on the target machine.

```
(root@kali)-[/home/kali]
# nmap -script http-vuln-cve2017-5638 10.0.0.102

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-19 17:44 EST
Stats: 0:00:09 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 42.25% done; ETC: 17:44 (0:00:12 remaining)
Nmap scan report for WIN-SFA18PMNQFM.phub.net.cable.rogers.com (10.0.0.102)
Host is up (0.0015s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsapi
MAC Address: 08:00:27:00:DD:1B (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 17.63 seconds
```

Fig.11

Fig.12 illustrates the content of the http-vuln-cve2017-5638.nse script and provides an example of the expected output or a similar result when executing the script with Nmap against a target machine.

```
1 description = [[
2 Detects whether the specified URL is vulnerable to the Apache Struts
3 Remote Code Execution Vulnerability (CVE-2017-5638).
4 ]]
5
6 local http = require "http"
7 local shortport = require "shortport"
8 local vulns = require "vulns"
9 local stdnse = require "stdnse"
10 local string = require "string"
11 local rand = require "rand"
12
13 --
14 -- @usage
15 -- nmap -p <port> --script http-vuln-cve2017-5638 <target>
16 --
17 -- @output
18 -- PORT      STATE SERVICE
19 -- 80/tcp    open  http
20 -- | http-vuln-cve2017-5638:
21 -- |   VULNERABLE
22 -- |   Apache Struts Remote Code Execution Vulnerability
23 -- |   State: VULNERABLE
24 -- |   IDs:   CVE:CVE-2017-5638
25 -- |
26 -- |   Disclosure date: 2017-03-07
27 -- |   References:
28 -- |     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-5638
29 -- |     https://cwiki.apache.org/confluence/display/WW/S2-045
30 -- |     http://blog.talosintelligence.com/2017/03/apache-0-day-exploited.html
31 -- |
32 -- @args http-vuln-cve2017-5638.method The HTTP method for the request. The default method is "GET".
33 -- @args http-vuln-cve2017-5638.path The URL path to request. The default path is "/".
34
```

Fig.12

Conclusion

This project successfully explored the methods of network and website scanning using Nmap, Wireshark, and NSE scripts to identify vulnerabilities and analyze the behavior of intrusion detection systems (IDS) and firewalls.

Through practical demonstrations and testing, key evasion techniques such as **Packet Fragmentation**, **IP Address Spoofing**, and **Source Port Manipulation** were implemented to evaluate their effectiveness in bypassing security mechanisms.

Wireshark was instrumental in capturing and analyzing network traffic, showcasing how these evasion techniques could manipulate packet behavior to avoid detection. **Nmap**, combined with the **NSE scripting engine**, provided an efficient way to scan for hosts, services, and vulnerabilities, enabling a deeper understanding of how attackers could exploit weaknesses.

Custom scripts like `http-headers.nse`, `dns-brute.nse`, and `http-vuln-cve2017-5638.nse` proved useful for uncovering subdomains, analyzing headers, and detecting specific vulnerabilities.

By leveraging tools like **Kali Linux**, a Windows test environment, and virtual machines, the project provided a controlled yet realistic platform for conducting tests. Target websites such as `test.vulnweb.com` and `hackthissite.com` offered practical environments to safely explore vulnerabilities without ethical or legal implications. In summary, this project demonstrated the importance of understanding network vulnerabilities and the functionality of IDS and firewalls. By learning how to detect and evade these systems, cybersecurity professionals can strengthen defensive measures, making networks more resilient against potential attacks.

Reference

- James Bogert, 2022. Nmap Firewall Evasion Techniques:
<https://linuxsecurity.com/features/nmap-firewall-evasion-techniques>

The background is a solid blue color. In the top-left corner, there is a series of white lines that curve and branch out, resembling a circuit board or data paths. In the bottom-left corner, there is a stylized globe composed of a network of white dots connected by thin lines, representing a global network. In the bottom-right corner, there is another series of white lines that curve and branch out, mirroring the design in the top-left.

THANK
YOU